

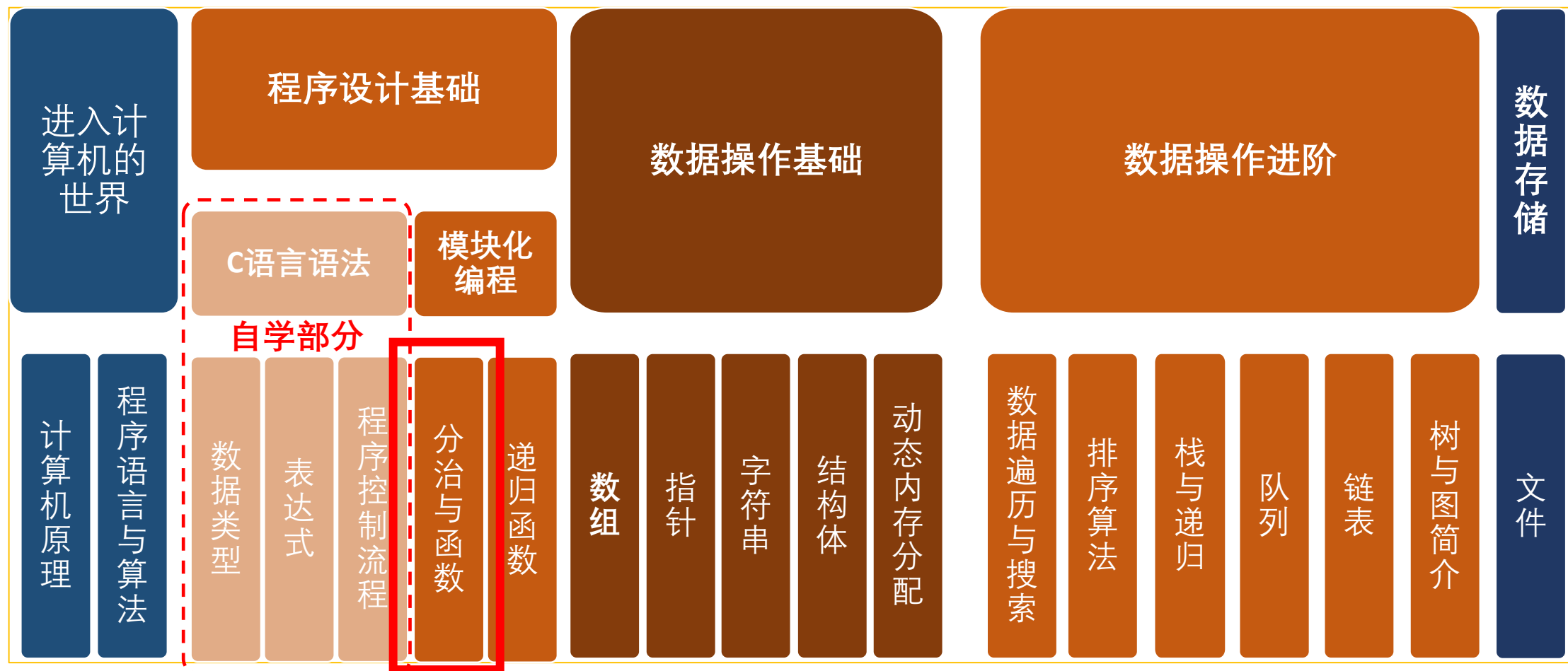


计算思维与实践

哈尔滨工业大学（深圳）
计算机科学与技术学院
大数据技术中心
张保权

课程内容安排

2



程序设计思想与数据操作方法融会贯通，内容由浅入深

第四讲 分治与函数-编程思维转变

3

自底向上的程序设计方法



自顶向下的程序设计方法

第四讲 学习内容

4

- 4.1 分而治之与信息隐藏
- 4.2 函数定义
- 4.3 函数的参数传递与返回值
- 4.4 变量的作用域与存储类



第四讲 学习内容

5

- 4.1 分而治之与信息隐藏
- 4.2 函数定义
- 4.3 函数的参数传递与返回值
- 4.4 变量的作用域与存储类



4.1分而治之与信息隐藏-从生活中的问题求解谈起

6

- 管理学观点认为工作必须分工，各司其职

Problem: 准备早餐 (Prepare a Breakfast)

金枪鱼三明治

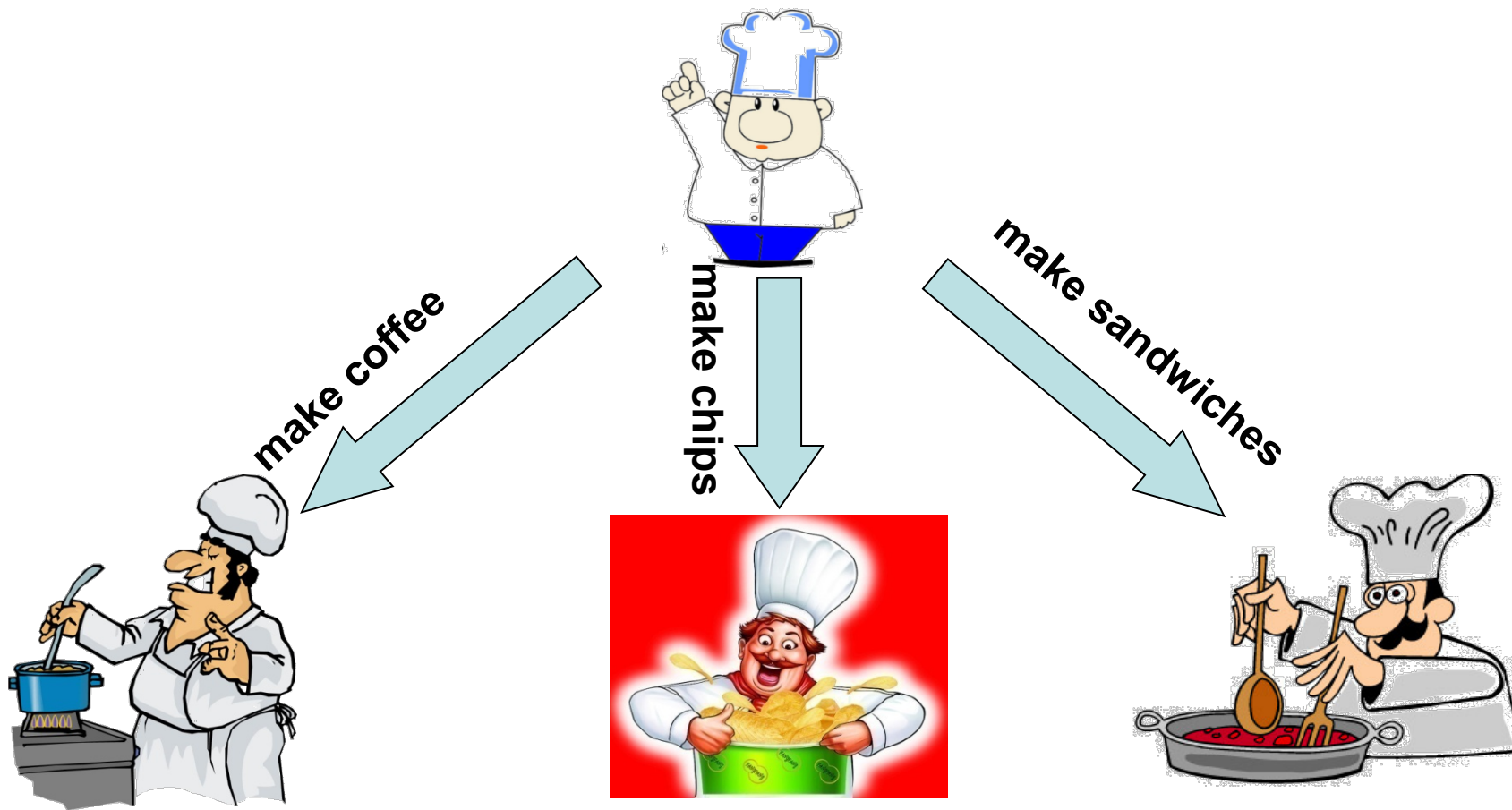
薯条

咖啡



4.1分而治之与信息隐藏-从生活中的问题求解谈起

7



分工明确，各司其职，并行工作

4.1分而治之与信息隐藏

8

- 分工合作的思想，在程序设计里也适用
- 假如将所有的功能都塞到一个`main()`中？
- 假如系统提供的函数`printf()`用10行代码替换，那么你编过的程序会成什么样子？
- 一个函数中适合放多少行代码？
 - 1986年IBM在OS/360的研究结果：多数有错误的函数大于500行
 - 1991年对148,000行代码的研究表明：小于143行的函数比更长的函数更容易维护



4.1分而治之与信息隐藏-分而治之

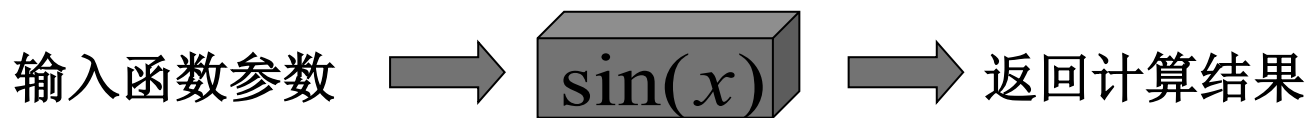
9

- 分而治之(Divide and Conquer, Wirth, 1971)
 - 把一个复杂的问题分解为若干个简单的问题，提炼出公共任务，把不同的功能分解到不同的模块中，逐个解决
 - 复杂问题求解的基本方法
 - 模块化编程的基本思想

4.1分而治之与信息隐藏-信息隐藏

10

- 信息隐藏（Information Hiding, Parnas, 1972）
 - 把函数内的具体操作细节对外界隐藏起来
 - 对于函数的使用者，无需知道函数内部如何运作，只了解其输入输出即可
 - 尽管汇编语言里有“子程序”，但高级语言中的函数真正体现了“封装”的思想
 - 它通过参数和返回值与外界交流，通过参数获取外部信息，函数接口必须定义清楚

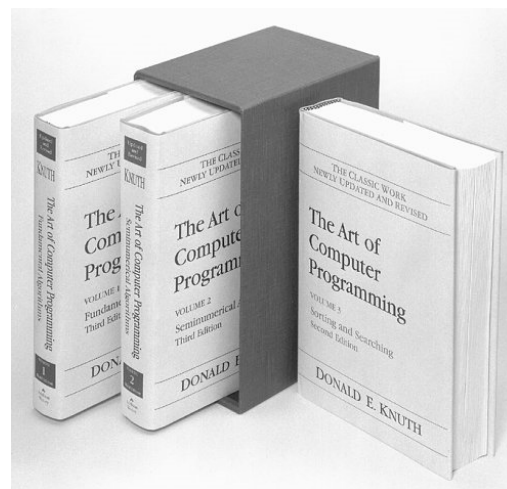


4.1分而治之与信息隐藏-设计艺术

11

■ 程序设计的艺术—“The Art of Computer Programming”

- Donald E. Knuth
- 清华大学出版社（英），国防工业出版社（中）
- 算法设计艺术——程序的灵魂
- 结构设计艺术——程序的骨架和肉体
 - 优雅的结构给人以美的享受
 - 模块化（Parnas, 1972）
 - 结构化（Structural）
 - 面向对象（Object-Oriented）
 - 面向组件（Component-Oriented）
 - 面向智能体（Agent-Oriented）……



第四讲 学习内容

12

- 4.1 分而治之与信息隐藏
- 4.2 函数定义
- 4.3 函数的参数传递与返回值
- 4.4 变量的作用域与存储类

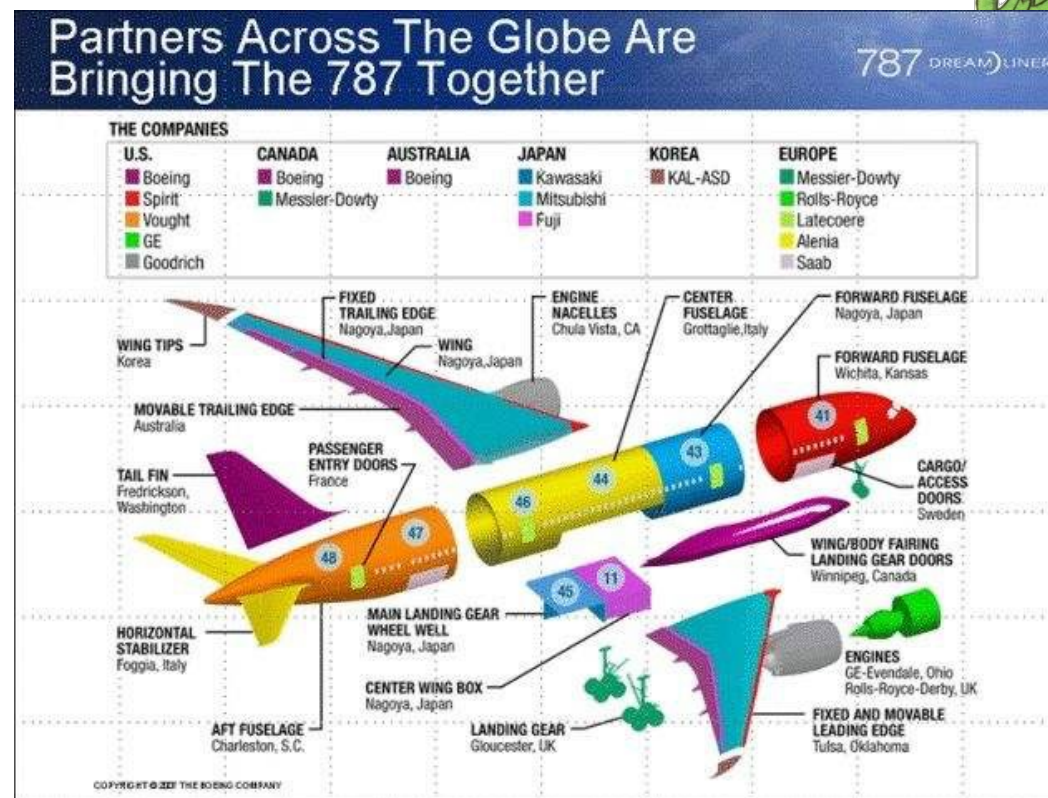


4.2 函数的定义

13

- 把编程比做制造一台机器，函数就好比其零部件
 - * 可将这些“零部件”单独设计、调试、测试好，用时拿出来装配，再总体调试
 - * 这些“零部件”可以是自己设计制造/别人设计制造/现成的标准产品

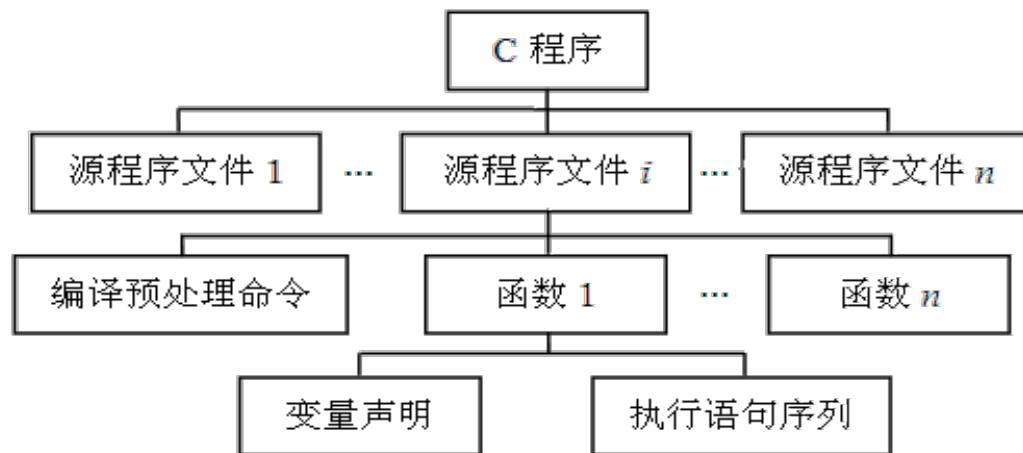
何谓函数？



4.2 函数的定义

14

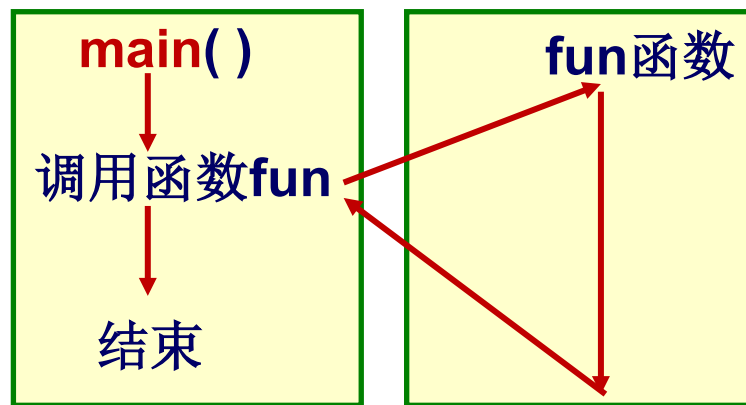
- 函数（Function）是C中模块化编程的最小单位
 - * 可把每个函数看作一个模块（Module）
 - * 若干相关的函数也可合并成一个模块
- C程序的结构
 - * 一个C程序由一个或多个源程序文件组成
 - * 一个源程序文件由一个或多个函数组成



4.2.1 函数的定义-函数的分类

15

- 函数生来都是平等的，互相独立的，没有高低贵贱和从属之分
 - `main()`稍微特殊一点点
 - C程序的执行从`main`函数开始
 - 调用其他函数后流程回到`main`函数
 - 在`main`函数中结束整个程序的运行



4.2.1 函数的定义-函数的分类

16

- 标准库函数
 - ANSI/ISO C定义的标准库函数
 - ◆符合标准的C语言编译器必须提供这些函数
 - ◆函数的行为也要符合ANSI/ISO C的定义
 - 第三方库函数
 - ◆由其他厂商自行开发的C语言函数库
 - ◆不在标准范围内，能扩充C语言的功能（图形、数据库等）
- 自定义函数
 - 用户自己定义的函数
 - ◆包装后，也可成为函数库，供别人使用

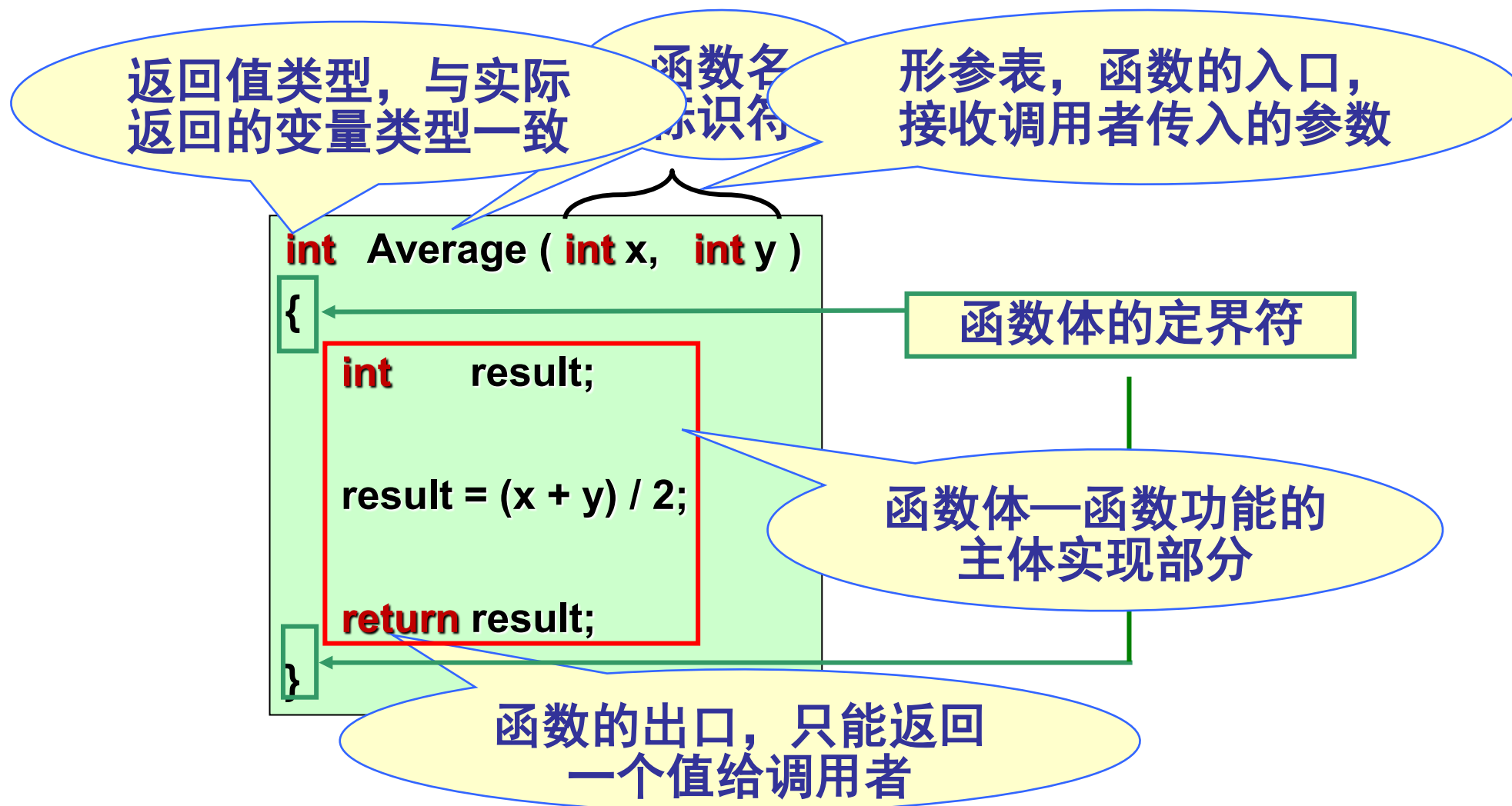
4.2.2 函数的定义-数学中的函数与程序设计中的函数

	数学中的函数		程序设计中的函数
数学表达式	$y = \sin(x)$	C语言表达式	$y = \sin(x)$
函数名	sin	函数名	sin
自变量	x	函数的参数	x
因变量	y	函数的返回值	y
功能	主要是计算	功能	不局限于计算，还有判断推理



4.2.3 函数的定义-函数的结构

18



4.2.3 函数的定义-函数的结构

19

表示无返回值
省略默认为int

表示无参数
, 可省略

```
void DisplayMenu ( void )  
{  
    printf("1. input");  
    printf("2. output");  
    printf("0. exit");  
    ...  
    return ;  
}
```

不符合C标准
, 意味着永
远不会在程
序终止前检
测程序状态

return后无任何表
达式或不写

```
void main()  
{  
    .....  
}
```

```
int main()  
{  
    .....  
    return 0;  
}
```

4.2.4 函数的定义-函数接口的注释说明

20

```
/* 函数功能: 实现××××功能  
   函数参数: 参数1, 表示××  
              参数2, 表示××  
   函数返回值: ×××××  
*/
```

返回值类型 函数名(形参表)

```
{  
  
    ...  
    return 表达式;  
}
```

```
/* 函数功能: 用迭代法计算 n!  
   函数入口参数: 整型变量 n 表示阶乘的阶数  
   函数返回值: 返回 n! 的值  
*/
```

```
long Fact(int n)  
{  
    int i;  
    long result = 1;  
    for (i=2; i<=n; i++)  
    {  
        result *= i;  
    }  
    return result;  
}
```

4.2.5 函数的定义-函数命名规则

21

- Linux/UNIX风格的函数名命名
 - **function_name**
- Windows风格的函数名命名
 - 用大写字母开头、大小写混排的单词组合而成
 - **FunctionName**
- 变量名形式
 - “名词”或者“形容词+名词”, 如oldValue与newValue等
- 函数名形式
 - “动词”或者“动词+名词”（动宾词组）, 如GetMax()等

第四讲 学习内容

22

- 4.1 分而治之与信息隐藏
- 4.2 函数定义
- 4.3 函数的参数传递与返回值
- 4.4 变量的作用域与存储类



4.3.1 向函数传递值和从函数返回值-函数调用

23

- 调用者通过**函数名**调用函数
- 函数无返回值时，单独作为一个函数调用语句

```
int main()
{
    ...
    DisplayMenu();
    ...
    return 0;
}
```

```
void DisplayMenu ( void )
{
    printf("1. input");
    printf("2. output");
    printf("0. exit");
    ...
    return ;
}
```

Function
Call?



4.3.1 向函数传递值和从函数返回值-函数调用

24

- 调用者通过函数名调用函数
- 有返回值时，可放到一个赋值表达式语句中

```
int main()
{
    int a=12, b=24, ave;
    ...
    ave = Average(a, b);
    ...
    return 0;
}
```

```
int Average(int x, int y)
{
    int result;

    result = (x + y) / 2;

    return result;
}
```

Function
Call?



4.3.1 向函数传递值和从函数返回值-函数调用

25

- 调用者通过**函数名**调用函数
- 有返回值时，可放到一个**赋值表达式语句**中
- 还可放到一个**函数调用语句**中，作为**另一个函数的参数**

```
int main()
{
    int a=12, b=24;
    ...
    printf("%d\n", Average(a, b));
    ...
    return 0;
}
```

```
int Average(int x, int y)
{
    int result;

    result = (x + y) / 2;

    return result;
}
```

4.3.1 向函数传递值和从函数返回值-函数调用

26

函数定义时的参数，形式参数（Parameter），简称**形参**
函数调用时的参数，实际参数（Argument），简称**实参**

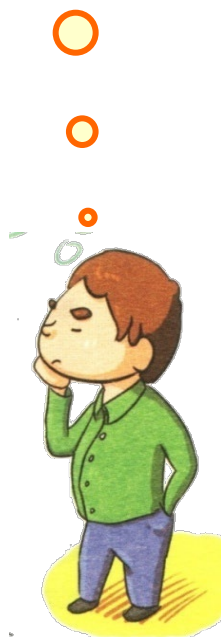
函数的
参数？

```
int main()
{
    int a=12, b=24;
    ...
    printf("%d\n", Average(a, b));
    ...
    return 0;
}
```

```
int Average(int x, int y)
{
    int result;

    result = (x + y) / 2;

    return result;
}
```

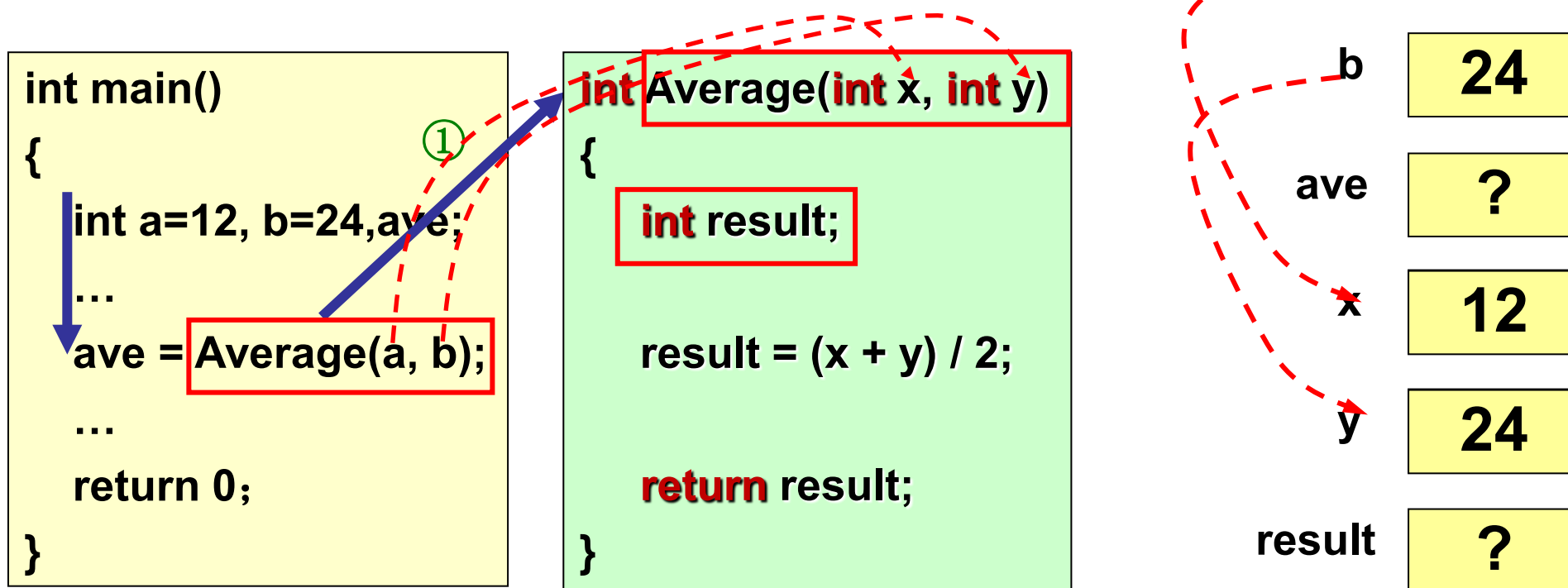


4.3.1 向函数传递值和从函数返回值-函数调用

27

■ 每次执行函数调用时

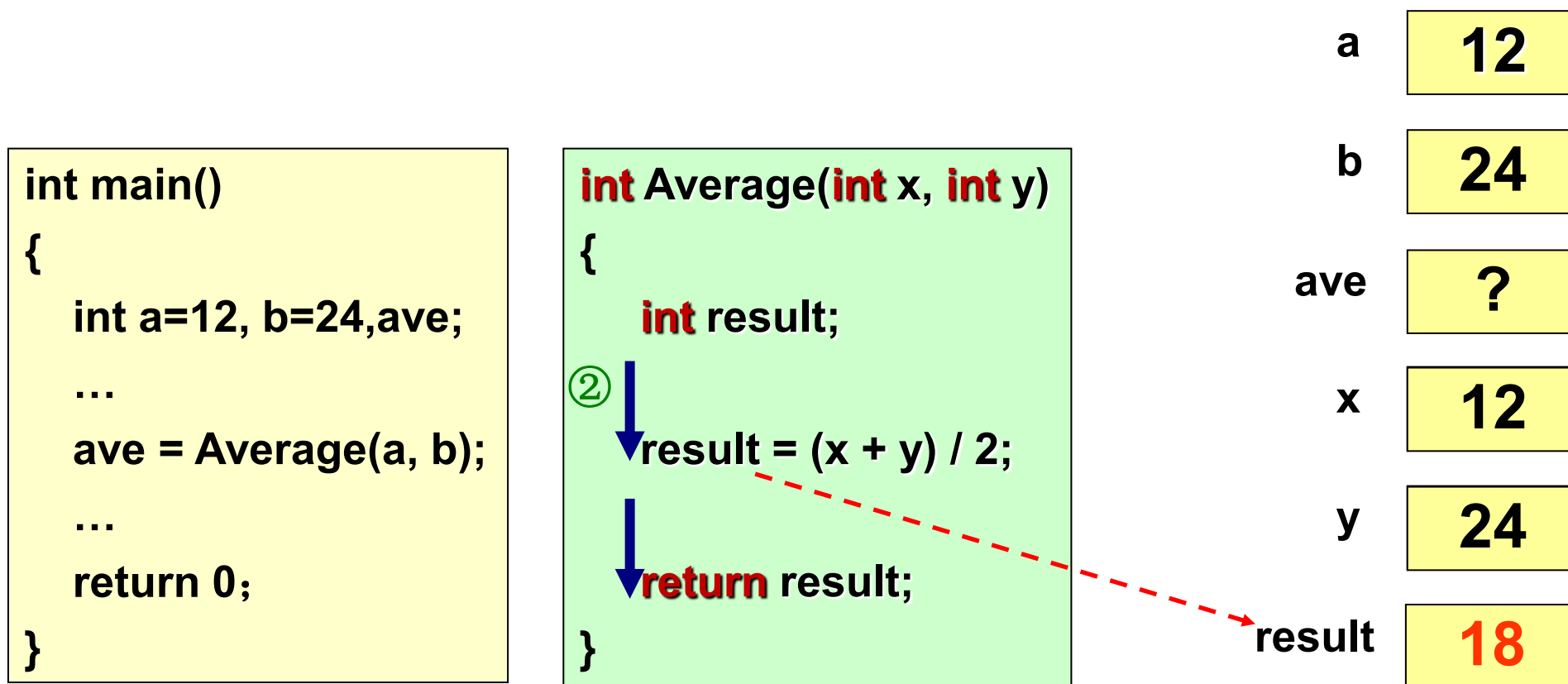
- 现场保护并为函数的内部变量（包括形参）分配内存
- 把实参值复制给形参，**单向传值**（实参→形参）
- 实参与形参数目一致，类型匹配（否则类型自动转换）



4.3.1 向函数传递值和从函数返回值-函数调用

28

- 执行函数内的语句
- 当执行到return语句或}时，从函数退出

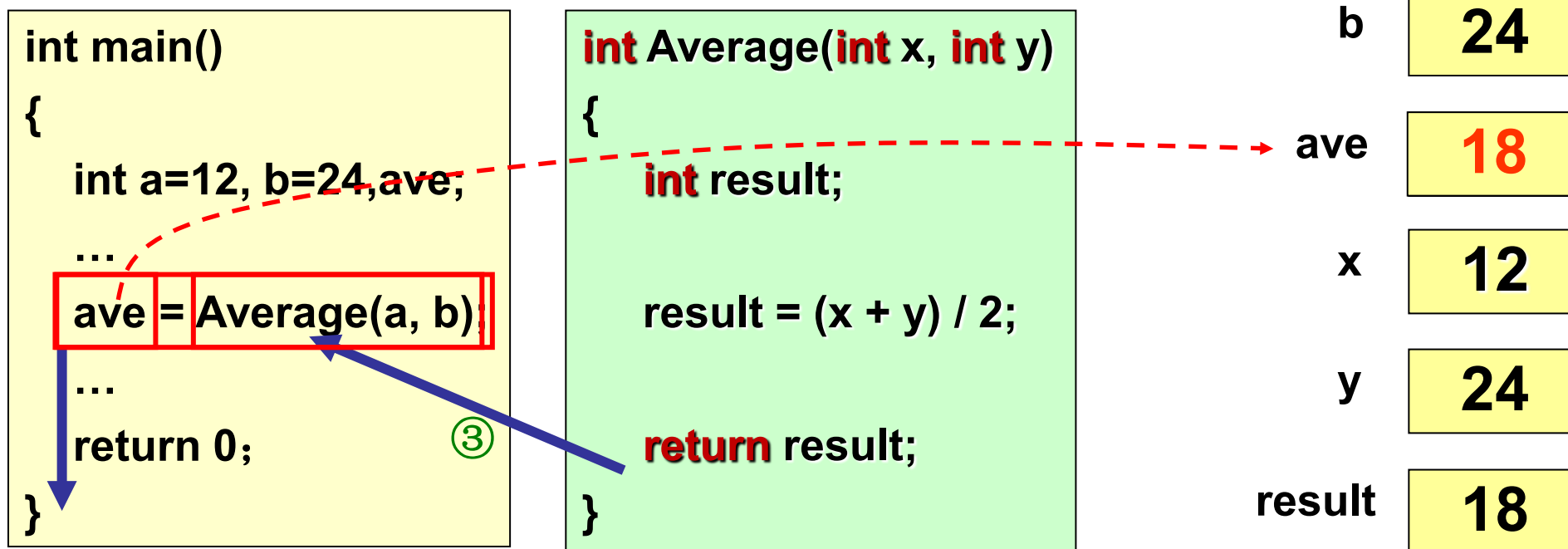


4.3.1 向函数传递值和从函数返回值-函数调用

29

■ 从函数退出时

- 根据函数调用栈中保存的返回地址，返回到当次函数调用的地方
- 程序控制权交给调用者，返回值作为函数调用表达式的值
- 收回分配给函数内所有变量（包括形参）的内存



【例4.1】 计算整数n的阶乘n!

30

```
long Fact(int n) /* 函数定义 */
{
    int i;
    long result = 1;
    for (i=2; i<=n; i++)
    {
        result *= i;
    }
    return result;
}
```

i	每次循环前result	每次循环后result
2	1	1*2
3	(1*2)	(1*2)*3
4	(1*2*3)	(1*2*3)*4
...	...	...
n	(1*2*3*4*...)	(1*2*3*4*...)*n

监视窗 (Watches)

4.3.2 向函数传递值和从函数返回值-函数原型

31

- 函数原型（Function Prototype）
 - 调用函数前先声明返回值类型、函数名和形参类型
 - 有助于编译器对函数参数类型的匹配检查

```
#include <stdio.h>
long Fact(int n);
int main()
{
    int m;
    long ret;
    printf("Input m:");
    scanf("%d", &m);
    ret = Fact(m);
    printf("%d! = %ld\n", m, ret);
    return 0;
}
```

```
long Fact(int n)
{
    int i;
    long result = 1;
    for (i=2; i<=n; i++)
    {
        result *= i;
    }
    return result;
}
```

4.3.2 向函数传递值和从函数返回值-函数原型

32

函数定义	函数原型
指函数功能的确立	对函数名、返回值类型、形参类型进行声明
有函数体	不包括函数体
是完整独立的单位	是一条语句，以分号结束，只起声明作用
编译器做实事	编译器对声明的态度是“我知道了”
分配内存，把函数装入内存	不申请内存，只保留一个引用，执行程序链接时，将正确的内存地址链接到那个引用上



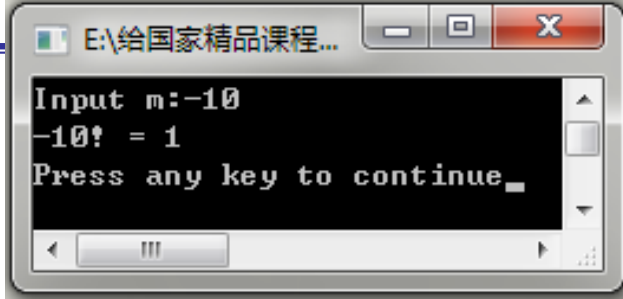
4.3.3 向函数传递值和从函数返回值-函数封装

33

- 函数封装（Encapsulation）
 - 外界对函数的影响仅限于入口参数
 - 函数对外界的影响仅限于一个返回值和数组、指针形参

```
#include <stdio.h>
long Fact(int n);
int main()
{
    int m;
    long ret;
    printf("Input m:");
    scanf("%d", &m);
    ret = Fact(m);
    printf("%d! = %ld\n", m, ret);
    return 0;
}
```

```
long Fact(int n)
{
    int i;
    long result = 1;
    for (i=2; i<=n; i++)
    {
        result *= i;
    }
    return result;
}
```



传入负数
实参
会怎样?



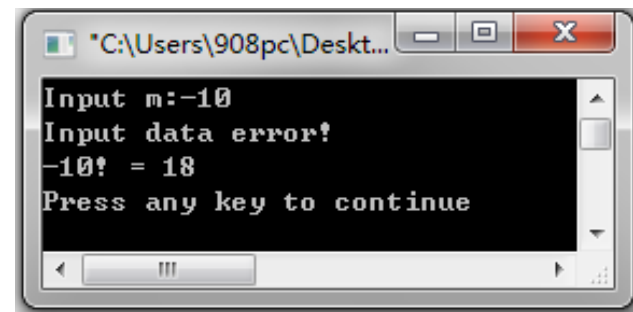
【例4.2】 计算整数n的阶乘n!

34

- 如何增强程序的健壮性，使函数具有遇到非法数据输入时避免出错的能力？
 - 在函数的入口处，检查输入参数的合法性

```
/* 函数功能：用迭代法计算 n! */
long Fact(int n)
{
    int i;
    long result = 1;
    if (n < 0)
    {
        printf("Input data error!\n");
    }
    else
    {
        for (i=2; i<=n; i++)
            result *= i;
        return result;
    }
}
```

```
long Fact(int n)
{
    int i;
    long result = 1;
    for (i=2; i<=n; i++)
    {
        result *= i;
    }
    return result;
}
```



缺陷：n<0时未指定返回值

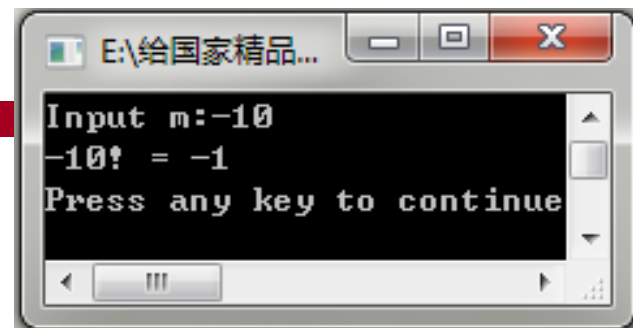
printf函数的返回值是其输出的字符个数

warning C4715: 'Fact' : not all control paths return a value

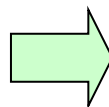
【例4.2】 计算整数n的阶乘n!

35

- 处理非法情况时，函数无返回值，如何修改？
 - 返回一个特定数字



```
/* 函数功能：用迭代法计算 n! */
long Fact(int n)
{
    int i;
    long result = 1;
    if (n < 0)
    {
        printf("Input data error!\n");
    }
    else
    {
        for (i=2; i<=n; i++)
            result *= i;
        return result;
    }
}
```



```
long Fact(int n)
{
    int i;
    long result = 1;
    if (n < 0)
    {
        return -1;
    }
    else
    {
        for (i=2; i<=n; i++)
            result *= i;
        return result;
    }
}
```

【例4.2】 计算整数n的阶乘n!

36

- 主函数如何修改?
 - 增加对函数返回值的检验

```
#include <stdio.h>
long Fact(int n);
int main()
{
    int m;
    long ret;
    printf("Input m:");
    scanf("%d", &m);
    ret = Fact(m);
    if (ret == -1)          /* 增加对函数返回值的检验 */
        printf("Input data error!\n");
    else
        printf("%d! = %ld\n", m, ret);
    return 0;
}
```



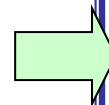
```
long Fact(int n)
{
    int i;
    long result = 1;
    if (n < 0)
    {
        return -1;
    }
    else
    {
        for (i=2; i<=n; i++)
            result *= i;
        return result;
    }
}
```

【例4.3】 计算整数n的阶乘n!

37

- 改成无符号整型，传入负数实参Fact()会返回-1吗？

```
#include <stdio.h>
unsigned long Fact(unsigned int n);
int main()
{
    int m;
    long ret;
    printf("Input m:");
    scanf("%d", &m);
    ret = Fact(m);
    if (ret == -1)          /* 增加对函数返回值的检验 */
        printf("Input data error!\n");
    else
        printf("%d! = %ld\n", m, ret);
    return 0;
}
```



```
unsigned long Fact(unsigned int n)
{
    unsigned int i;
    unsigned long result = 1;
    if (n < 0)
    {
        return -1;
    }
    else
    {
        for (i=2; i<=n; i++)
            result *= i;
        return result;
    }
}
```



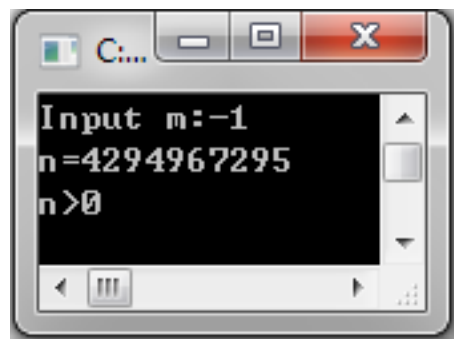
【例4.3】 计算整数n的阶乘n!

38

- 改成无符号整型，传入负数实参Fact()会返回-1吗？
 - 存在死代码的原因何在？

插入打印语句printf("n=%u", n);和其他验证信息

```
unsigned long Fact(unsigned int n)
{
    unsigned int i;
    long result = 1;
    printf("n=%u\n", n);
    if (n < 0)
    {
        printf("n<0");
        return -1;
    }
    else
    {
        printf("n>0");
        for (i=2; i<=n; i++)
            result *= i;
        return result;
    }
}
```



```
unsigned long Fact(unsigned int n)
{
    unsigned int i;
    unsigned long result = 1;
    if (n < 0)
    {
        return -1;
    }
    else
    {
        for (i=2; i<=n; i++)
            result *= i;
        return result;
    }
}
```

CB warning: comparison of unsigned expression < 0 is always false

【例4.3】 计算整数n的阶乘n!

39

- 去除冗余代码
- 如何保证不会传入负数实参?



```
#include <stdio.h>
unsigned long Fact(unsigned int n);
int main()
{
    int m;
    do{
        printf("Input m(m>0):");
        scanf("%d", &m);
    }while (m<0);
    printf("%d! = %lu\n", m, Fact(m));
    return 0;
}
```

```
unsigned long Fact(unsigned int n)
{
    unsigned int i;
    unsigned long result = 1;
    for (i=2; i<=n; i++)
        result *= i;
    return result;
}
```

Input m(m>0): -1 ✓

Input m(m>0): 10 ✓

10! = 3628800

✓ 【例4.4】 编写计算组合数的程序

40

$$C_m^k = \frac{m!}{k!(m-k)!}$$

```
#include <stdio.h>
unsigned long Fact(unsigned int n);
int main()
{
    int m, k;
    double p;
    do{
        printf("Input m,k (m>=k>0):");
        scanf("%d,%d", &m, &k);
    }while (m<k||m<0||k<0);
    p = (double)Fact(m) / (Fact(k) * Fact(m-k));
    printf("p = %.0f\n", p);
    return 0;
}
```

```
unsigned long Fact(unsigned int n)
{
    unsigned int i;
    unsigned long result = 1;
    for (i=2; i<=n; i++)
        result *= i;
    return result;
}
```

函数复用

- ① Input m,k (m>=k>0): 3,2✓
p = 3
- ② Input m,k (m>=k>0): 2,3✓
Input m,k (m>=k>0): 3,3✓
p = 1
- ③ Input m,k (m>=k>0): -2,-4✓
Input m,k (m>=k>0): 4,2✓
p = 6

自学内容-断言

41

- 测试一个条件并可能使程序终止

```
#include <assert.h>
```

```
void assert(int expression);
```

— `expression`为真，无声无息；为假，中断程序

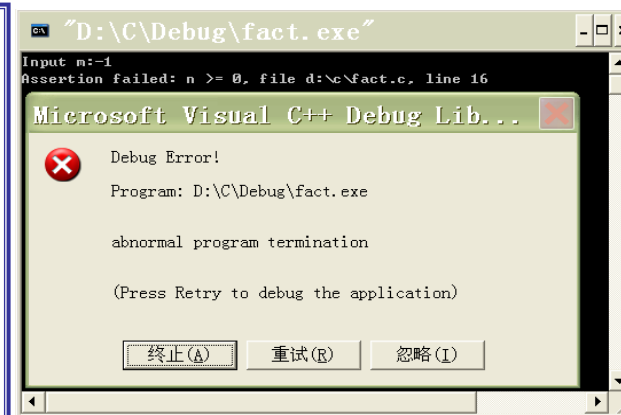
```
unsigned long Fact(unsigned int n)
{
    unsigned int i;
    unsigned long result = 1;
    assert(n >= 0);
    for (i=2; i<=n; i++)
        result *= i;
    return result;
}
```

测试 $n \geq 0$ 假设的正确性，即测试 $n < 0$ 确实是不可能发生的

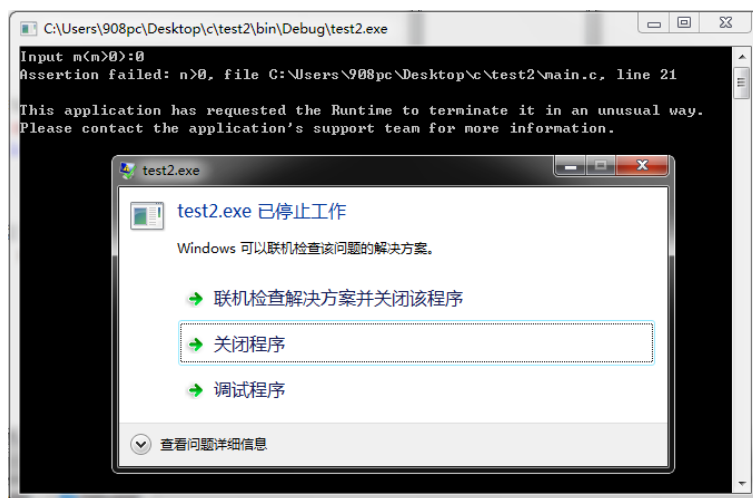
自学内容-断言

42

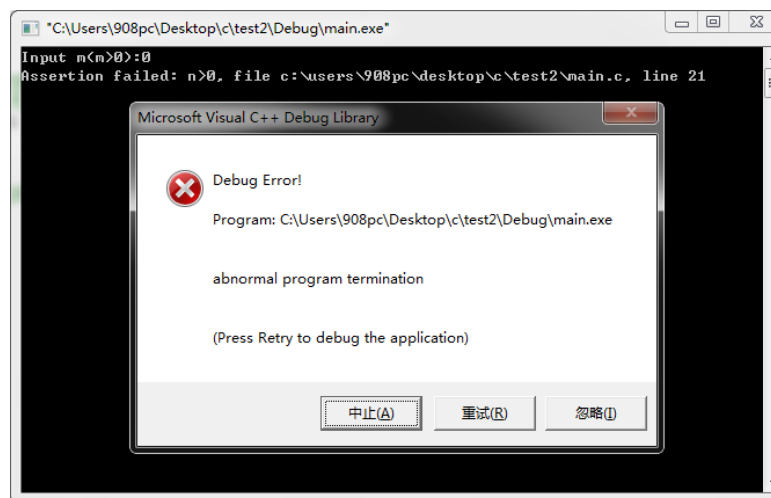
```
unsigned long Fact(unsigned int n)
{
    unsigned int i;
    unsigned long result = 1;
    assert(n>0);
    for (i=2; i<=n; i++)
        result *= i;
    return result;
}
```



XP



Win7 CB



VC

这里n值有可能为0?



- 考虑使用断言的几种情况
 - 检查程序中的各种假设的正确性
 - 证实或测试某种不可能发生的状况确实不会发生
- 仅用于调试程序，不能作为程序的功能
 - Debug版有效
 - Release版失效

第四讲 学习内容

44

- 4.1 分而治之与信息隐藏
- 4.2 函数定义、函数调用、函数原型
- 4.3 函数的参数传递与返回值
- 4.4 变量的作用域与存储类



4.4.1 变量的作用域和存储类型-变量的作用域

45

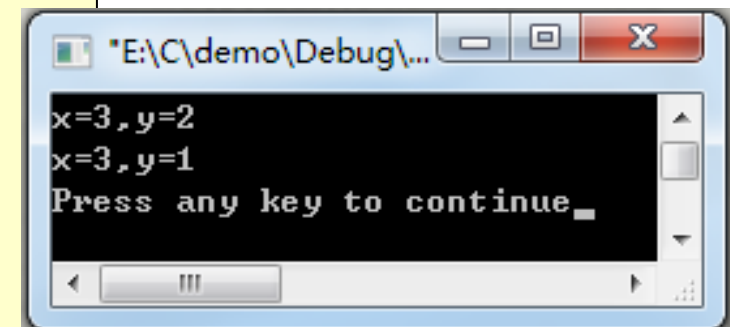
- 变量的作用域（Scope）
 - 指在源程序中定义变量的位置及其能被读写访问的范围，分为：
 - 局部变量（Local Variable）
 - 在语句块内（函数、复合语句）定义的变量
 - 全局变量（Global Variable）
 - 在所有函数之外定义的变量

4.4.1 变量的作用域和存储类型-变量的作用域

46

- 局部变量（Local Variable）的作用域
 - 有效范围仅为该语句块（函数，复合语句）
 - 仅能由语句块内的语句访问，退出语句块时释放内存，不再有效

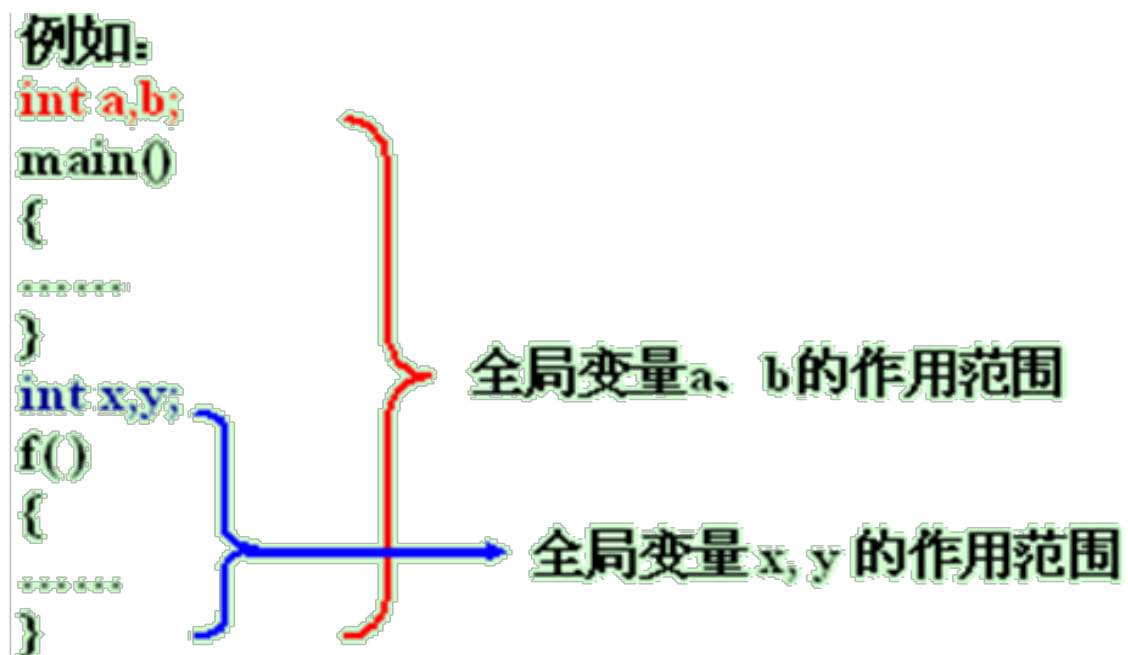
```
int main()
{
    int x = 1, y = 1;
    {
        int y = 2;
        x = 3;
        printf("x=%d,y=%d\n", x, y);
    }
    printf("x=%d,y=%d\n", x, y);
    return 0;
}
```



4.4.1 变量的作用域和存储类型-变量的作用域

47

- 全局变量（Global Variable）的作用域
 - 有效范围是从定义变量的位置开始，到本程序结束



4.4.1 变量的作用域和存储类型-变量的作用域

48

- 变量同名的情况

并列语句块内各自定义（不同作用域）的变量同名：

- 互不干扰（形参和实参的作用域不同）



```
#include <stdio.h>
void Function(int x, int y);
int main()
{
    int x = 1;
    int y = 2;
    Function(x, y);
    printf("x=%d,y=%d\n", x, y);
    return 0;
}
```

```
void Function(int x, int y)
{
    x = 2;
    y = 1;
    printf("x=%d,y=%d\n", x, y);
}
```

A screenshot of a Windows command prompt window. The title bar shows the path "D:\C\demo\abc\bin\Debug\a...". The window contains two lines of output: "x=2,y=1" on the first line and "x=1,y=2" on the second line. The window has standard Windows window controls (minimize, maximize, close) in the top right corner.

4.4.1 变量的作用域和存储类型-变量的作用域

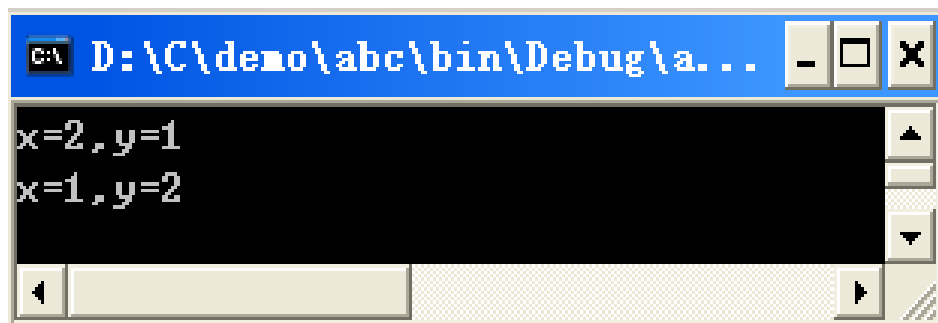
49

- 变量同名的情况

局部变量与全局变量同名:

- 局部变量屏蔽全局变量

```
#include <stdio.h>
void Function();
int x = 1;
int y = 2;
int main()
{
    Function();
    printf("x=%d,y=%d\n",x,y);
    return 0;
}
```



```
void Function()
{
    int x, y;
    x = 2;
    y = 1;
    printf("x=%d,y=%d\n",x,y);
}
```

4.4.1 变量的作用域和存储类型-变量的作用域

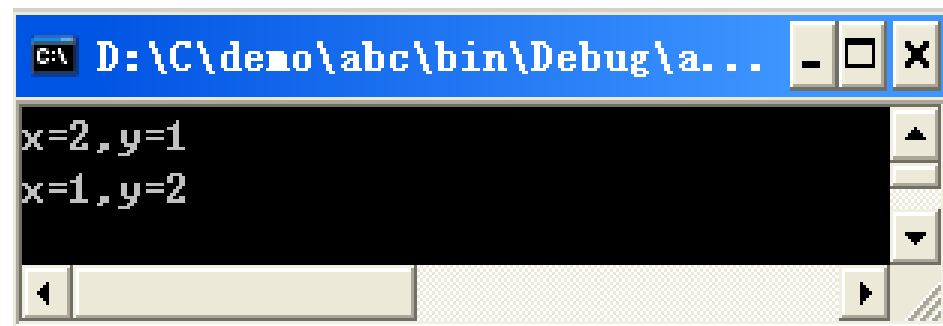
50

- 假如变量同名.....

- 只要作用域不同:

新的声明屏蔽旧的声明

```
#include <stdio.h>
void Function(int x, int y);
int x = 1;
int y = 2;
int main()
{
    Function(x, y);
    printf("x=%d,y=%d\n",x,y);
    return 0;
}
```



```
G:\ D:\C\demo\abc\bin\Debug\a...
x=2,y=1
x=1,y=2
```

```
void Function(int x, int y)
{
    x = 2;
    y = 1;
    printf("x=%d,y=%d\n",x,y);
}
```

4.4.1 变量的作用域和存储类型-变量的作用域

51

- 假如同名变量出现在同一个作用域中？
 - 编译器将无法区分。编译器只能区分不同作用域中的同名变量
- 编译器是如何区分不同作用域的同名变量的呢？
 - 硬件无法识别作用域的划分。硬件只知道操作内存地址



4.4.1 变量的作用域和存储类型-变量的作用域

52

- 编译器如何区分不同作用域的同名变量？
 - 变量名与内存地址的关系
 - 一个变量名能代表两个不同的值，仅当它能代表两个不同的内存地址
 - 作用域划分与内存地址映射
 - 编译器通过将同名变量映射到不同的内存地址来实现作用域的划分
 - 局部变量与全局变量的内存分配
 - 局部变量和全局变量被分配的内存区域不同，因而内存地址也不同
 - 形参与实参的作用域和内存独立性
 - 形参和实参的作用域、内存地址不同，所以形参值的改变不会影响实参



4.4.1 变量的作用域和存储类型-变量的作用域

53

- 全局变量使用场景：
 - 当多个函数必须共享同一个变量
 - 少数几个函数必须共享大量变量

使用全局变量，函数间的数据交换更容易，更高效



4.4.1 变量的作用域和存储类型-变量的作用域

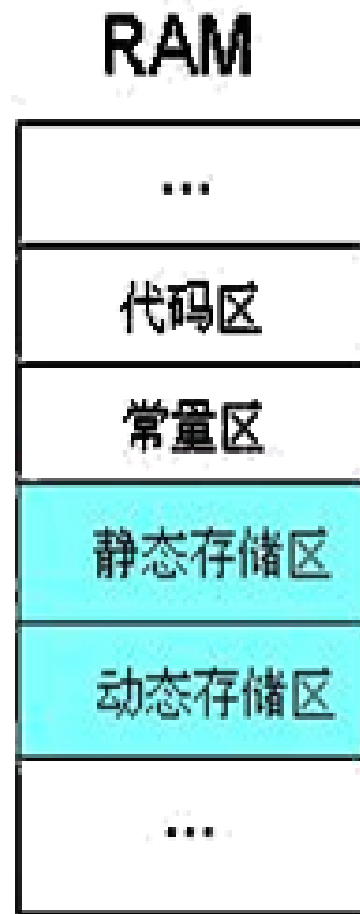
54

- 尽量少用全局变量
- 多数情况下，通过形参和返回值进行数据交流比共享全局变量的方法更好
 - 不能实现信息隐藏，谁都可改写它，很难确定谁改写了它，就像处理聚集很多人的晚会上的谋杀案很难缩小嫌疑犯范围一样
 - 很难在其他程序中复用，依赖全局变量的函数不是“独立”的，为在另一程序中使用它，不得不带上函数所需的全局变量
 - 修改程序时，若改变全局变量（如类型），则需检查同一文件中的每个函数，以确认该变化对函数的影响程度

4.4.2 变量的作用域和存储类型-变量的存储类型

55

- 内存中供用户使用的存储空间可分为4个部分
 - “静态” 含义
 - ◆ 表示事情发生在程序构建的编译时和链接时，而非程序实际开始运行的载入时和运行时
 - “动态” 含义
 - ◆ 表示事情发生在程序载入时和运行时
 - 变量的存储类型
 - ◆ 在内存中存储（编译器为变量分配内存）的方式
 - ◆ 静态存储方式，是指在程序运行期间分配固定的存储空间的方式



4.4.2 变量的作用域和存储类型-变量的存储类型

56

- 存储类型决定变量的生存期（Lifetime）
 - ◆ 何时“生”，何时“灭”
 - ◆ 静态分配的变量在整个程序的生命周期内全程占据内存

静态存储区中的变量：与**程序** “共存亡”
动态存储区中的变量：与**语句块** “共存亡”



4.4.2 变量的作用域和存储类型-变量的存储类型

57

- 如何声明变量的存储类型？
存储类型 **数据类型** **变量名**;
- C存储类型关键字
 - **auto**（自动变量）
 - **static**（静态变量）
 - **extern**（外部变量）
 - 编译器并不对其分配内存，只是表明“我知道了”
 - **register**（寄存器变量）

4.4.2 变量的作用域和存储类型-变量的存储类型

58

- 自动变量——动态局部变量（缺省类型）
 - auto** 数据类型 变量名；
 - 进入语句块时**自动**申请内存，退出时**自动**释放内存
 - **离开函数，值就消失**
- 静态变量
 - static** 数据类型 变量名；
 - 从程序运行起占据内存，程序退出时释放内存
 - **离开函数，值仍保留**
 - 包括：静态局部变量，静态外部变量
 - 生存期相同，作用域不同（语句块内，文件内）

【例4.11】利用静态变量计算整数n的阶乘n!

59

```
1  #include <stdio.h>
2  long Func(int n);
3  int main()
4  {
5      int i, n;
6      printf("Input n:");
7      scanf("%d", &n);
8      for (i=1; i<=n; i++)
9      {
10         printf("%d!", i);
11     }
12     return 0;
13 }
14 long Func(int n)
15 {
16     static long p = 1; /*定义静态变量*/
17     p = p * n;
18     return p;
19 }
```

静态变量在编译时赋值，仅初始化一次，变量的值可保存到下次进入函数，使函数具有记忆功能

Input n:10↵

1! = 1↵

2! = 2↵

3! = 6↵

4! = 24↵

5! = 120↵

6! = 720↵

7! = 5040↵

8! = 40320↵

9! = 362880↵

10! = 3628800↵

【例4.11】利用静态变量计算整数n的阶乘n!

60

若静态变量换成自动变量，
结果如何？

```
1  #include <stdio.h>
2  long Func(int n);
3  int main()
4  {
5      int i, n;
6      printf("Input n:");
7      scanf("%d", &n);
8      for (i=1; i<=n; i++)
9      {
10         printf("%d!", i);
11     }
12     return 0;
13 }
14 long Func(int n)
15 {
16     auto long p = 1; /*定义自动变量*/
17     p = p * n;
18     return p;
19 }
```

静态局部变量和全局变量自动初始化为0。自动变量不初始化时，值是随机值

Input n:10✓

1! = 1✓

2! = 2✓

3! = 3✓

4! = 4✓

5! = 5✓

6! = 6✓

7! = 7✓

8! = 8✓

9! = 9✓

10! = 10✓



4.4.2 变量的作用域和存储类型-变量的存储类型

61

- 寄存器变量

寄存器是CPU内部容量有限、但速度极快的存储器

register 类型名 变量名;

CPU

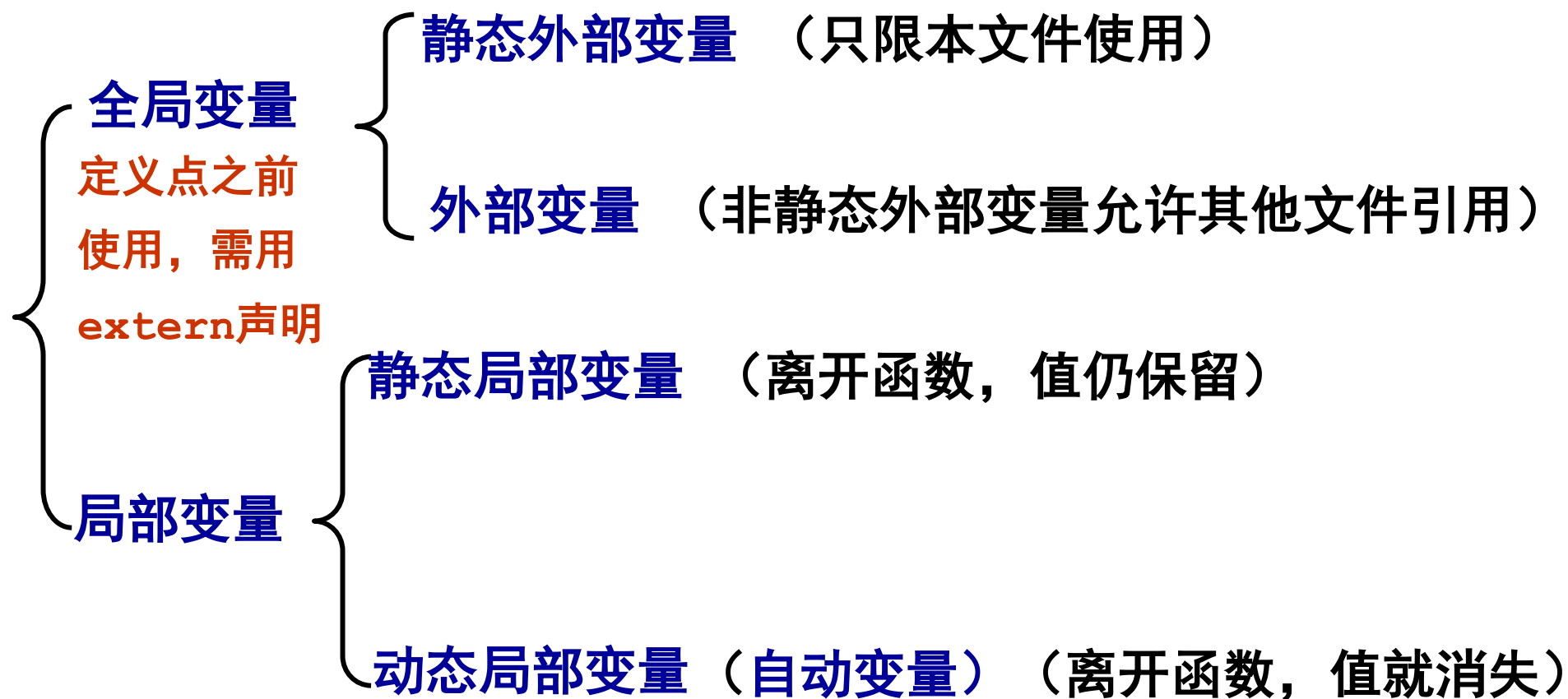
运算器+控制器

寄存器

- 适用于使用频率较高的变量，可使程序更小、执行速度更快
- 寄存器变量的生存期与程序“共存亡”
- 现代编译器有能力自动把普通变量优化为寄存器变量，并且可以忽略用户的指定，所以一般无需特别声明变量为**register**

4.4.3 变量的作用域和存储类型-总结

62



本讲小结

63

- 结构化编程的思想
- 函数定义
 - 函数结构：函数名、形参、返回值类型和函数体
 - 函数的分类，如标准库函数、第三方库函数和自定义函数
- 函数调用
 - 函数调用的流程 P24
- 变量的作用域
 - 内存地址与变量 P48
 - 全局变量与局部变量的对比 P58
- 变量的存储类型
 - 静态存储区与动态存储区 P51
 - 存储类型声明（四种） P53